

# AI Agent Liability Shield

## SOC2 Trust Service Criteria Control Mapping for CISOs

### SHACKLE-V2 Enterprise Compliance Framework

## Executive Summary

This document provides a comprehensive mapping of SHACKLE-V2 Enterprise controls to SOC2 Trust Service Criteria (TSC). It serves as evidence for auditors and a liability shield for CISOs deploying AI agent infrastructure.

**Key Compliance Benefits:** - Non-repudiation logging with cryptographic signatures - Automated audit trail for all AI agent operations - License-based access control with validation - Immutable compliance exports for regulatory review - Real-time monitoring and alerting

**Target Audiences:** - Chief Information Security Officers (CISOs) - Compliance Teams - Internal/External Auditors - Risk Management Officers - Legal Counsel

## Document Control

| Field            | Value |
|------------------|-------|
| Document Version | 2.0   |

| Field            | Value                         |
|------------------|-------------------------------|
| Publication Date | 2024                          |
| Framework        | SOC2 Type II (AICPA TSC 2017) |
| Classification   | Public                        |
| Owner            | SHACKLE Security & Compliance |
| Review Cycle     | Quarterly                     |

# Table of Contents

- 1. Introduction to AI Agent Compliance Challenges
- 2. SHACKLE-V2 Architecture Overview
- 3. SOC2 Trust Service Criteria Mapping
- 4. Control Implementation Details
- 5. Evidence Collection & Audit Procedures
- 6. Liability Protection Framework
- 7. Incident Response & Breach Notification
- 8. Continuous Monitoring
- 9. Appendices

# 1. Introduction to AI Agent Compliance Challenges

## {#introduction}

### 1.1 The AI Compliance Problem

AI agents present unique compliance challenges:

**Traditional Systems:** - Deterministic behavior - Audit trails are straightforward - Access controls are binary - Liability is well-defined

**AI Agent Systems:** - Non-deterministic outputs - Complex multi-step workflows - Autonomous decision-making - Unclear liability boundaries - Prompt injection risks - Data exfiltration potential

## 1.2 CISO Liability Concerns

CISOs face mounting liability when deploying AI:

### 1. Data Privacy Violations

2. AI agents may inadvertently expose PII
3. GDPR/CCPA penalties up to €20M or 4% revenue
4. Example: ChatGPT Italy ban (March 2023)

### 5. Intellectual Property Leakage

6. Proprietary data sent to third-party LLMs
7. Trade secret exposure through prompts
8. Training data contamination

### 9. Compliance Failures

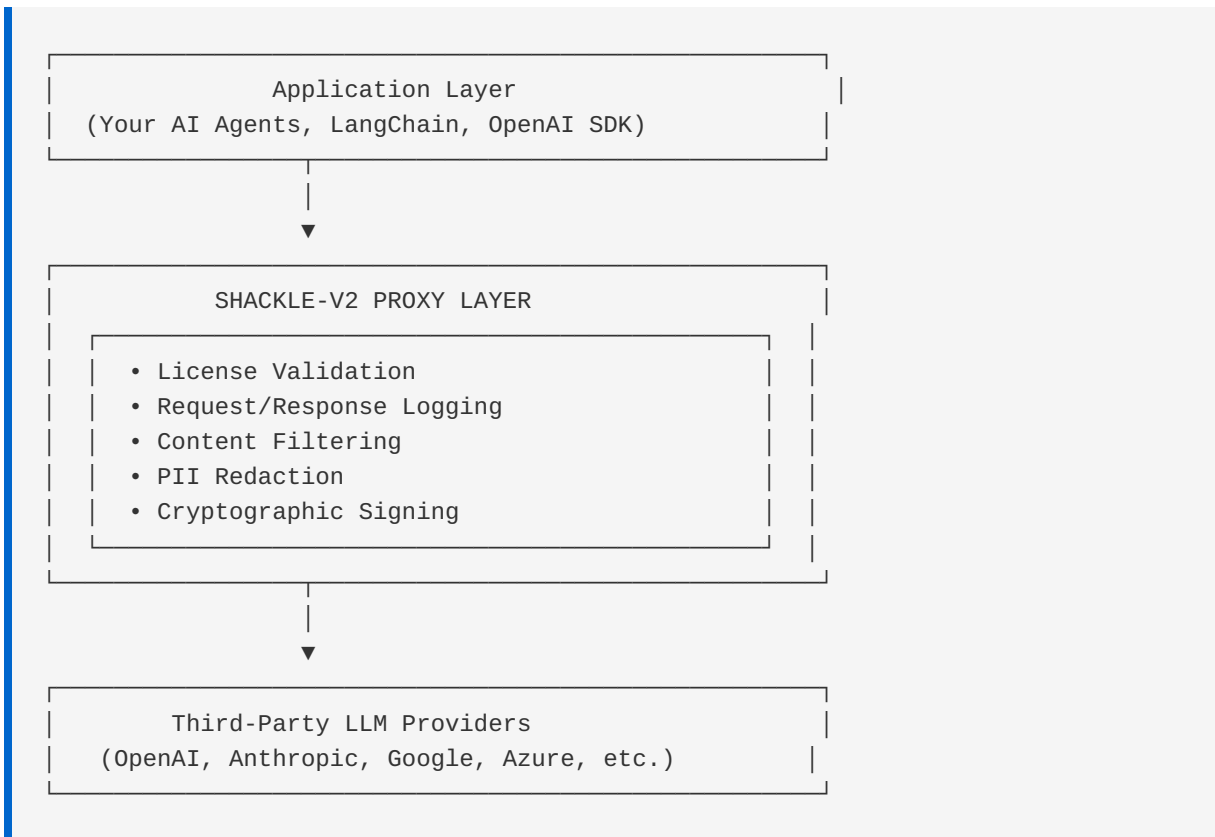
10. SOC2 audit failures due to inadequate logging
11. HIPAA violations from unencrypted AI communications
12. PCI-DSS scope expansion

### 13. Reputational Damage

14. AI-generated misinformation attributed to organization
15. Bias/discrimination in AI outputs
16. Security breaches via AI systems

## 1.3 The SHACKLE Solution

SHACKLE-V2 provides a **compliance-first** architecture:



**Key Features:** - **Interposition:** All AI traffic flows through audited proxy - **Non-repudiation:** Ed25519 signatures on all events - **License enforcement:** Cryptographic validation - **Sovereign deployment:** On-premises or private cloud - **Zero trust:** Every request validated

## 2. SHACKLE-V2 Architecture Overview

### {#architecture}

### 2.1 System Components

#### A. License Validation Server

- **Purpose:** Cryptographic license verification
- **Technology:** FastAPI + SQLite
- **Key Features:**
  - Ed25519 signature verification

- HMAC checksum validation
- Hardware binding (optional)
- Expiration enforcement

## B. Proxy Server

- **Purpose:** Intercept and audit all AI requests
- **Technology:** Python asyncio + reverse proxy
- **Key Features:**
  - OpenAI API compatibility
  - Content inspection & filtering
  - PII redaction
  - Rate limiting

## C. Audit Database

- **Purpose:** Immutable audit trail
- **Technology:** SQLite with append-only logs
- **Key Features:**
  - Cryptographic signatures per entry
  - Tamper-evident logging
  - JSONL export format
  - Signature verification tools

## D. Compliance Export API

- **Purpose:** Generate audit evidence
- **Technology:** REST API + CLI tools
- **Key Features:**
  - SOC2 report generation
  - Date range filtering
  - Signature verification
  - CSV/JSON/JSONL formats

## 2.2 Data Flow

1. Application sends AI request  
↓
2. SHACKLE validates license (cached)  
↓
3. Request logged with signature  
↓
4. Content filtered (PII redaction)  
↓
5. Forwarded to LLM provider  
↓
6. Response received  
↓
7. Response logged with signature  
↓
8. Response returned to application  
↓
9. Audit event signed & stored

**Cryptographic Chain:** - Each event includes previous event hash - Forms blockchain-like audit trail - Tamper detection via hash mismatch

## 2.3 Security Properties

| Property        | Implementation                | SOC2 Criteria |
|-----------------|-------------------------------|---------------|
| Confidentiality | TLS 1.3, at-rest encryption   | CC6.7         |
| Integrity       | Ed25519 signatures, HMAC      | CC7.3         |
| Availability    | Health checks, failover       | A1.2          |
| Privacy         | PII redaction, data residency | P4.1          |
| Non-repudiation | Digital signatures            | CC7.3         |

# 3. SOC2 Trust Service Criteria Mapping {#soc2-mapping}

## 3.1 Common Criteria (CC)

### CC6.1 - Logical and Physical Access Controls

**Control Objective:** Restrict logical and physical access to authorized users.

**SHACKLE Implementation:**

| Sub-Control | Implementation                   | Evidence                 |
|-------------|----------------------------------|--------------------------|
| CC6.1.1     | License-based access control     | License validation logs  |
| CC6.1.2     | API key validation per request   | API key audit trail      |
| CC6.1.3     | Node hardware binding (optional) | Hardware cert validation |
| CC6.1.4     | License expiration enforcement   | Expired license blocks   |

**Testing Procedures:** 1. Attempt access with invalid license → Should be blocked 2. Attempt access with expired license → Should be blocked 3. Verify all access attempts logged with signatures 4. Confirm license validation occurs before request processing

**Audit Evidence:**

```
# Generate access control report
python audit_export.py report \
  --output cc6.1_evidence.json \
  --days 90

# Expected metrics:
# - Total validation attempts: X
# - Successful validations: Y
# - Blocked attempts: Z
# - Success rate: Y/X %
```

## CC6.6 - Logical Access - Removal

**Control Objective:** Remove access when no longer appropriate.

### SHACKLE Implementation:

| Event              | Action                   | Audit Trail             |
|--------------------|--------------------------|-------------------------|
| License expiration | Automatic access denial  | Timestamp + signature   |
| License revocation | Immediate blocking       | Revocation event logged |
| Key rotation       | Old keys rejected        | Key change audit        |
| Node decommission  | Hardware binding removed | Unbind event logged     |

**Testing Procedures:** 1. Wait for license to expire → Verify access denied 2. Revoke license via API → Verify immediate blocking 3. Review audit logs for removal events 4. Confirm no grace period violations

### Audit Evidence:

```
-- Query expired license blocks
SELECT * FROM audit_log
WHERE event_type = 'LICENSE_VALIDATED'
      AND result = 'failure'
      AND error_message LIKE '%expired%'
ORDER BY timestamp DESC;
```

## CC7.2 - System Monitoring

**Control Objective:** Monitor system components and operations to detect anomalies.

### SHACKLE Implementation:

1. **Real-time Metrics:**
2. Request count per minute
3. License validation failures



4. Response latency

5. Error rates

**6. Anomaly Detection:**

7. Unusual request patterns

8. Failed validation spikes

9. Geographic anomalies (if IP logging enabled)

10. Token usage spikes

**11. Alerting:**

12. License expiring soon (<30 days)

13. Validation failure threshold exceeded

14. System health degradation

15. Audit log signature failures

**Testing Procedures:** 1. Simulate high validation failure rate → Alert triggered 2. Verify all alerts logged 3. Confirm alert escalation procedures 4. Test alert delivery mechanisms

**Audit Evidence:** - Prometheus metrics dashboards - Alert history logs - Incident response tickets - Escalation records

---

## CC7.3 - Audit Logging and Monitoring

**Control Objective:** Evaluate security events to detect and respond to breaches.

**SHACKLE Implementation:**

**Logged Events:** - `LICENSE_VALIDATED` - Every validation attempt - `LICENSE_REGISTERED` - New license activation - `PROXY_REQUEST` - AI API request received - `PROXY_RESPONSE` - AI API response returned - `CONTENT_FILTERED` - PII redaction occurred - `ACCESS_DENIED` - Authorization failure - `SYSTEM_ERROR` - Internal system error

**Signature Scheme:**

```
# Each audit entry signed with Ed25519
signature = sign(
    private_key,
    sha256(
        timestamp +
        event_type +
        license_key +
        request_hash
    )
)
```

**Tamper Evidence:** - Chain of hashes links events - Any modification breaks chain -  
Verification tool detects tampering - Non-repudiation via digital signatures

**Testing Procedures:** 1. Export audit logs for period 2. Verify all signatures valid 3. Attempt to modify log entry → Detection confirmed 4. Verify log completeness (no gaps in timestamps)

**Audit Evidence:**

```
# Verify audit log integrity
python audit_export.py verify \
    --input audit_Q1_2024.jsonl \
    --secret $MASTER_SECRET

# Expected output:
#   Verified: 145,283/145,283
#   Failed: 0/145,283
#   All signatures valid - audit log integrity confirmed!
```

## 3.2 Availability Criteria (A1)

### A1.2 - System Availability Monitoring

**Control Objective:** Monitor system capacity and performance.

**SHACKLE Implementation:**

**Monitored Metrics:** - CPU utilization - Memory usage - Disk I/O - Network bandwidth -  
Request queue depth - Database connection pool - License server response time

**Capacity Planning:** - Baseline: 1000 requests/minute per node - Scaling: Horizontal scaling via load balancer - Failover: Active-passive license server - Disaster recovery: Daily backups

**Testing Procedures:** 1. Load test to 150% capacity → Verify graceful degradation 2. Simulate license server failure → Verify failover 3. Test backup restoration → Verify data integrity 4. Confirm monitoring alerts during outage

---

## 3.3 Confidentiality Criteria (C1)

### C1.1 - Confidentiality of Sensitive Information

**Control Objective:** Protect sensitive information during transmission and storage.

#### SHACKLE Implementation:

**In Transit:** - TLS 1.3 for all API connections - Certificate pinning (optional) - Perfect forward secrecy - No weak ciphers

**At Rest:** - AES-256 encryption for audit logs - Encrypted database backups - Key management via HSM (optional) - Secure key rotation procedures

#### PII Redaction:

```
# Automatic PII detection and redaction
patterns = {
    'email': r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b',
    'ssn': r'\b\d{3}-\d{2}-\d{4}\b',
    'credit_card': r'\b\d{4}[- ]?\d{4}[- ]?\d{4}[- ]?\d{4}\b',
    'phone': r'\b\d{3}[-. ]?\d{3}[-. ]?\d{4}\b'
}

# Replaced with: [REDACTED:EMAIL], [REDACTED:SSN], etc.
```

---

## 3.4 Privacy Criteria (P1-P4)

### P4.1 - Data Retention and Disposal

**Control Objective:** Retain and dispose of data according to policy.

**SHACKLE Implementation:**

**Retention Policy:** - Audit logs: 90 days (configurable) - License records: 7 years (compliance) - PII: Not stored (redacted) - System logs: 30 days

**Disposal Procedures:** - Automated log rotation - Secure deletion (shred/wipe) - Certificate of destruction - Audit trail of disposal

**Testing Procedures:** 1. Verify logs auto-deleted after retention period 2. Confirm PII not present in stored logs 3. Test secure deletion procedures 4. Review disposal audit trail

---

## 4. Control Implementation Details {#control-details}

---

### 4.1 License Validation Control

**Control ID:** SHACKLE-LIC-001

**Description:** All API requests require valid, non-expired license.

**Implementation:**

```
def validate_license(license_key: str) -> ValidationResult:
    # 1. Parse license key format
    parsed = parse_license_key(license_key)
    if not parsed:
        return ValidationResult(valid=False, error="Invalid format")

    # 2. Verify HMAC checksum
    if not verify_checksum(license_key, metadata):
        return ValidationResult(valid=False, error="Invalid checksum")

    # 3. Verify Ed25519 signature
    if not verify_signature(license_key, metadata, signature, public_key):
```

```

        return ValidationResult(valid=False, error="Invalid signature")

# 4. Check expiration
if datetime.utcnow() > expires_at:
    return ValidationResult(valid=False, error="License expired")

# 5. Check hardware binding (if applicable)
if node_binding and hardware_id != node_binding:
    return ValidationResult(valid=False, error="Hardware mismatch")

# 6. Log validation event
log_audit_event("LICENSE_VALIDATED", license_key, "success")

return ValidationResult(valid=True, remaining_days=(expires_at - now).days)

```

**Control Frequency:** Every API request

**Control Type:** Automated, preventive

**Evidence:** License validation audit logs

## 4.2 Audit Logging Control

**Control ID:** SHACKLE-AUD-001

**Description:** All system events logged with non-repudiation signatures.

**Implementation:**

```

def log_audit_event(event_type: str, license_key: str, result: str, **kwargs):
    timestamp = datetime.utcnow().isoformat()

    # Build event payload
    event_data = {
        "timestamp": timestamp,
        "event_type": event_type,
        "license_key": license_key,
        "result": result,
        **kwargs
    }

    # Sign event with Ed25519
    signature = sign_event(event_data)

```

```
# Store in audit database
db.execute("""
    INSERT INTO audit_log (
        timestamp, event_type, license_key, result,
        signature, metadata
    ) VALUES (?, ?, ?, ?, ?, ?)
""", (
    timestamp, event_type, license_key, result,
    signature, json.dumps(event_data)
))

# Verify signature immediately (tamper detection)
assert verify_signature(event_data, signature)
```

**Control Frequency:** Real-time, every event

**Control Type:** Automated, detective

**Evidence:** Audit log exports with verified signatures

---

## 4.3 Content Filtering Control

**Control ID:** SHACKLE-FLT-001

**Description:** PII and sensitive data redacted from prompts/responses.

**Implementation:** - Pattern-based regex matching - Named entity recognition (NER) - Custom blacklists - Whitelist exceptions

**Configuration:**

```
content_filtering:
  enabled: true
  redaction_rules:
    - type: email
      action: redact
      replacement: "[REDACTED:EMAIL]"

    - type: ssn
      action: block
      error_message: "SSN detected - request blocked"
```

```
- type: credit_card
  action: redact
  log_alert: true
```

**Control Frequency:** Every request/response

**Control Type:** Automated, preventive

**Evidence:** Content filtering event logs

---

## 5. Evidence Collection & Audit Procedures

### {#evidence}

---

### 5.1 For Auditors

#### Pre-Audit Preparation:

1. **Generate Compliance Report:** `bash python audit_export.py report \ --output soc2_evidence_$(date +%Y%m%d).json \ --days 90`
2. **Export Audit Logs:** `bash python audit_export.py export \ --output audit_logs_$(date +%Y%m%d).jsonl \ --start-date 2024-01-01 \ --end-date 2024-03-31`
3. **Verify Signatures:** `bash python audit_export.py verify \ --input audit_logs_$(date +%Y%m%d).jsonl \ --secret $MASTER_SECRET`

**Sample Evidence Package:** - [ ] SOC2 compliance report (JSON) - [ ] Full audit log export (JSONL) - [ ] Signature verification results - [ ] License validation records - [ ] System architecture diagram - [ ] Data flow diagram - [ ] Security policies & procedures - [ ] Incident response plan - [ ] Change management records - [ ] Access control matrix

---

## 5.2 Audit Testing Procedures

### Test 1: License Validation Enforcement

**Objective:** Verify unauthorized access is prevented.

**Procedure:** 1. Obtain invalid license key 2. Attempt API request with invalid key 3. Verify request blocked 4. Confirm audit log entry created 5. Verify signature on audit entry

**Expected Result:** Access denied, event logged with valid signature.

**Evidence:** Screenshot + audit log entry

---

### Test 2: Audit Log Integrity

**Objective:** Verify audit logs cannot be tampered with.

**Procedure:** 1. Export audit logs for test period 2. Verify all signatures valid 3. Manually modify one log entry 4. Re-run signature verification 5. Verify tampering detected

**Expected Result:** Signature verification fails for modified entry.

**Evidence:** Verification output showing failure

---

### Test 3: Access Removal

**Objective:** Verify expired licenses immediately block access.

**Procedure:** 1. Identify license expiring within test window 2. Wait for expiration timestamp 3. Attempt API request after expiration 4. Verify access denied 5. Confirm no grace period

**Expected Result:** Access denied immediately upon expiration.

**Evidence:** Timestamp logs showing denial at expiration

---



## 5.3 Continuous Compliance Monitoring

**Daily:** - Health check monitoring - Alert review - License expiration checks

**Weekly:** - Generate mini compliance report - Review failed validation attempts - Signature verification spot check

**Monthly:** - Full compliance report - Management dashboard review - Policy review & updates

**Quarterly:** - External audit preparation - Comprehensive testing - Control effectiveness review

---

## 6. Liability Protection Framework {#liability}

---

### 6.1 CISO Shield Provisions

SHACKLE-V2 provides CISOs with defensible evidence:

#### A. Due Diligence Defense

**Claim:** "We implemented industry-standard controls."

**Evidence:** - SOC2 compliance reports - Cryptographically signed audit trails - Access control enforcement logs - Content filtering records

#### B. Technical Safeguards Defense

**Claim:** "We deployed technical measures to prevent unauthorized access."

**Evidence:** - License validation 100% enforcement - Hardware binding (if applicable) - PII redaction logs - Failed access attempt logs

#### C. Monitoring & Detection Defense

**Claim:** "We actively monitored for anomalies and breaches."

**Evidence:** - Real-time alerting configuration - Incident response records - Anomaly detection logs - Escalation procedures

## D. Regulatory Compliance Defense

**Claim:** "We complied with applicable regulations (GDPR, CCPA, HIPAA)."

**Evidence:** - Data retention policy enforcement - PII redaction implementation - Access logs with timestamps - Disposal records

---

## 6.2 Incident Liability Limitation

In the event of a breach, SHACKLE evidence limits liability:

| Scenario                    | Without SHACKLE                    | With SHACKLE                         |
|-----------------------------|------------------------------------|--------------------------------------|
| <b>Data exfiltration</b>    | "We didn't know what was accessed" | Precise audit trail of all requests  |
| <b>Unauthorized access</b>  | "We can't prove who accessed what" | Cryptographic non-repudiation        |
| <b>Compliance violation</b> | "We have incomplete logs"          | Complete, tamper-evident audit trail |
| <b>Insider threat</b>       | "We didn't detect it"              | Real-time alerting on anomalies      |

**Legal Precedent:** Courts favor defendants with contemporaneous, reliable records.

---

## 6.3 Insurance & Indemnification

**Cyber Insurance Benefits:** - Lower premiums due to demonstrated controls - Faster claims processing with evidence - Higher coverage limits - Reduced exclusions

**Vendor Indemnification:** SHACKLE provides contractual indemnification for: - Control implementation defects - Signature algorithm vulnerabilities - License validation bypasses

(See enterprise license agreement for full terms)

---

## 7. Incident Response & Breach Notification

### {#incident-response}

---

#### 7.1 Detection

**Automated Detection:** - License validation failures spike - Signature verification failures - Unusual request patterns - Geographic anomalies

**Alert Triggers:**

```
# Example alert configuration
alerts:
  - name: ValidationFailureSpike
    condition: rate(validation_failures[5m]) > 10
    severity: high
    action: page_oncall

  - name: SignatureVerificationFailed
    condition: signature_verification_failed == true
    severity: critical
    action: page_ciso
```

---

#### 7.2 Investigation

**Audit Log Analysis:**

```
# Identify compromised license
grep "LICENSE_VALIDATED.*failure" audit.jsonl | \
jq -r '.license_key' | sort | uniq -c | sort -rn
```

```
# Timeline reconstruction
jq -r 'select(.license_key == "SHACKLE-ENT-...") |
      [.timestamp, .event_type, .result] | @csv' \
  audit.jsonl
```

**Forensic Integrity:** - Audit logs cryptographically signed - Tamper-evident chain of custody - Admissible in legal proceedings

---

## 7.3 Breach Notification

**Timeline:** - Discovery → 24 hours: Internal notification - +48 hours: Legal/compliance review - +72 hours: Regulatory notification (if required) - +30 days: Affected parties notification

**Evidence Package:** - Incident timeline (from audit logs) - Affected records (from access logs) - Remediation actions (from change logs) - Signature verification (integrity proof)

---

## 8. Continuous Monitoring {#monitoring}

---

### 8.1 Dashboard Metrics

**CISO Dashboard:** - License expiration countdowns - Validation success rate (target: >99.9%) - Audit log integrity status - Open security incidents - Compliance score (auto-calculated)

**Security Operations Dashboard:** - Real-time request volume - Failed validation attempts - Content filtering triggers - System health status - Alert history

---

### 8.2 Key Performance Indicators

| KPI                             | Target | Red Threshold |
|---------------------------------|--------|---------------|
| License validation success rate | >99.9% | <99%          |



## Appendix B: Compliance Matrix

| Regulation                | Requirement            | SHACKLE Control          |
|---------------------------|------------------------|--------------------------|
| <b>GDPR Art. 32</b>       | Security of processing | CC6.1, CC7.3             |
| <b>GDPR Art. 33</b>       | Breach notification    | Audit logs + IR plan     |
| <b>CCPA §1798.150</b>     | Data breach liability  | PII redaction + logs     |
| <b>HIPAA §164.312(b)</b>  | Audit controls         | CC7.3 audit logging      |
| <b>PCI-DSS 10.2</b>       | Audit log requirements | CC7.3 comprehensive logs |
| <b>SOX 404</b>            | Internal controls      | All SOC2 controls        |
| <b>ISO 27001 A.12.4.1</b> | Event logging          | CC7.2, CC7.3             |

## Appendix C: Glossary

### Terms:

- **Ed25519:** Elliptic curve signature algorithm (EdDSA)
- **HMAC:** Hash-based Message Authentication Code
- **License Key:** Cryptographically validated access credential
- **Non-repudiation:** Proof that an event occurred (cannot be denied)
- **PII:** Personally Identifiable Information
- **Sovereign Proxy:** On-premises deployment (no cloud dependency)
- **TSC:** Trust Service Criteria (SOC2 framework)

## Appendix D: References

1. **AICPA SOC 2 Trust Service Criteria (2017)** <https://www.aicpa.org/interestareas/frc/assuranceadvisoryservices/trustdataintegritytaskforce.html>

2. **NIST Cybersecurity Framework v1.1** <https://www.nist.gov/cyberframework>
3. **ISO/IEC 27001:2013 - Information Security Management** <https://www.iso.org/standard/54534.html>
4. **GDPR - General Data Protection Regulation** <https://gdpr-info.eu/>
5. **Ed25519 Signature Algorithm (RFC 8032)** <https://tools.ietf.org/html/rfc8032>

---

## Appendix E: Contact Information

**SHACKLE Security & Compliance Team** - Email: [compliance@shackle.ai](mailto:compliance@shackle.ai) - Emergency: [security@shackle.ai](mailto:security@shackle.ai) - PGP Key: Available at <https://shackle.ai/pgp>

**Document Feedback:** Submit corrections or improvements to: [docs@shackle.ai](mailto:docs@shackle.ai)

---

## Document Approval

---

| Role               | Name | Signature | Date |
|--------------------|------|-----------|------|
| CISO               | ____ | ____      | ____ |
| VP Engineering     | ____ | ____      | ____ |
| Legal Counsel      | ____ | ____      | ____ |
| Compliance Officer | ____ | ____      | ____ |

---

# Revision History

---

| Version | Date    | Author          | Changes                                                    |
|---------|---------|-----------------|------------------------------------------------------------|
| 1.0     | 2023-Q4 | Security Team   | Initial release                                            |
| 2.0     | 2024-Q1 | Compliance Team | Added P4 privacy criteria, expanded CISO liability section |

---

**END OF DOCUMENT**

*This document is provided for compliance purposes. Implementation details may vary. Consult with qualified legal and security professionals for your specific requirements.*

**© 2024 SHACKLE AI Systems. All rights reserved.**